KAVACH

Sanitation Agent

Deep Technical Explanation & Flowcharts

A beginner-friendly guide explaining what the agent does, how to implement it in Python, how the cleaning plan is created, how resources and time are estimated, and how the result is validated.

Purpose	Analyze sanitation requirements and produce a structured cleaning plan, resource estimate, duration estimate and task checklist.
Prototype stack	Python, FastAPI, Pydantic, Gemini API, deterministic rules, templates and in-memory sample data.
Key principle	Gemini proposes or adapts; Python calculates and enforces rules; Pydantic validates the data structure.
Scope	Sanitation Agent only. Other agents are not designed in this document.

Important: The source diagram specifies four outputs—cleaning plan, resources required, estimated time and task checklist. Implementation details, sample categories, rules and numeric examples in this guide are proposed design choices and must be confirmed by the KAVACH team.

1. Understand the agent in the easiest language

Imagine a report says: “The dustbin near the main gate is overflowing. Garbage is scattered around it and the walking path is partly blocked.” The Sanitation Agent turns this complaint into a practical plan. It decides which cleaning activities are needed, what resources the job may require, how long it might take and what steps must be checked.

Think of the agent as a planning assistant—not the worker who performs the cleaning. It creates instructions; it must not claim that cleaning has already happened.

The four outputs from the diagram:

• Cleaning plan: the overall objective and ordered work steps.

• Resources required: estimated workers, tools, protective equipment, vehicles and supplies.

• Estimated time: duration based on quantities, task rules and explicit assumptions.

• Task checklist: small steps with dependencies and completion criteria.

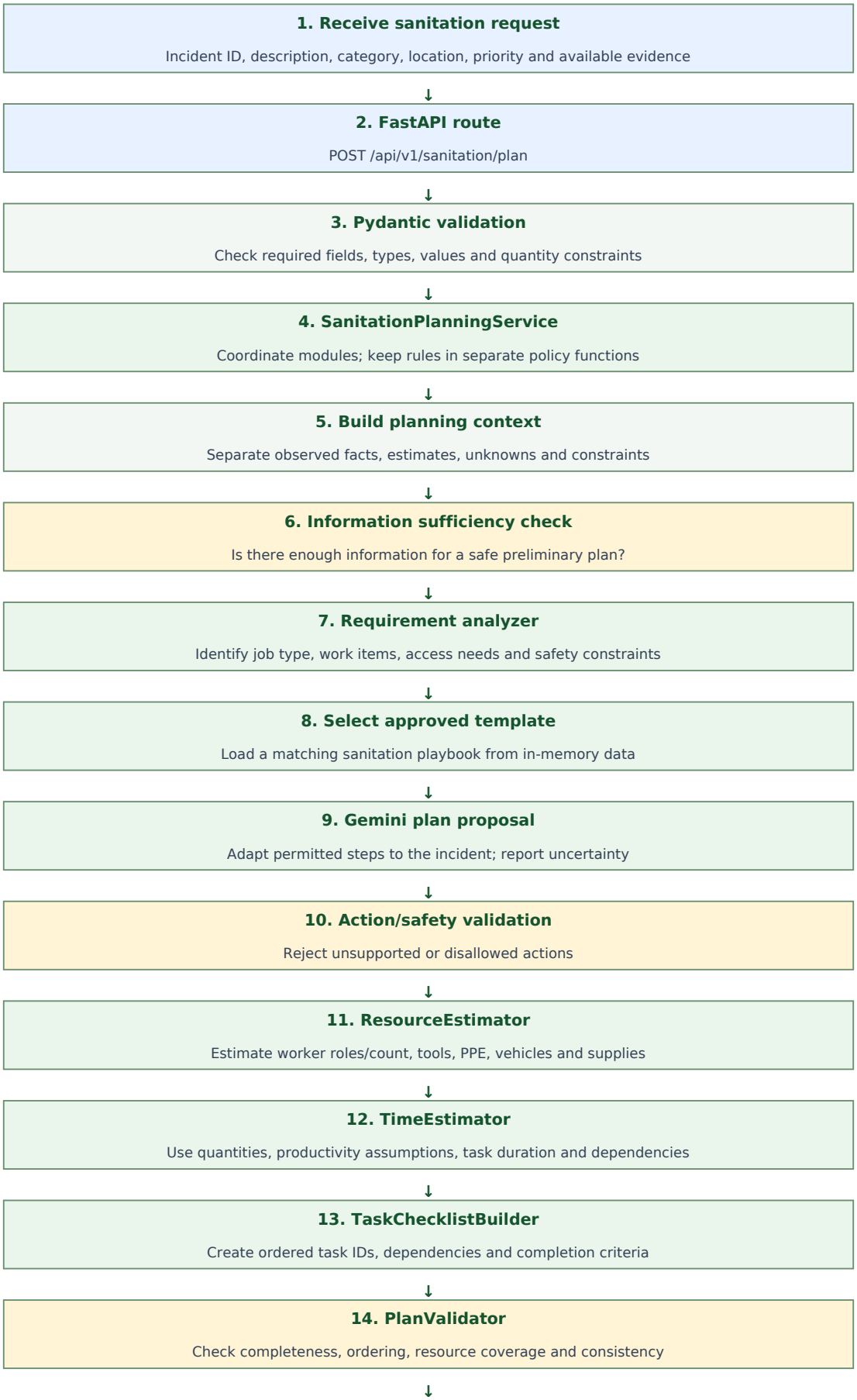
2. Recommended prototype architecture

Component	Plain-language job	Technical responsibility
FastAPI	Front door to the agent.	Receive request and return JSON via HTTP.
Pydantic	Form checker.	Validate request/response field types, required fields and enums.
Python service	Coordinator.	Call modules in the correct order and handle controlled failure paths.
Gemini API	Language-understanding helper.	Interpret complaint details and propose/adapt steps within supplied constraints.
Rule and template modules	Approved playbook.	Supply approved actions, safety constraints, resource assumptions and task templates.
Estimators	Calculators.	Calculate resources and duration from configured inputs and assumptions.
In-memory repository	Temporary sample shelf.	Hold test templates and sample productivity rules for the prototype.

For the first prototype, use a regular Python service before introducing a workflow framework. A framework is optional; clear module boundaries and predictable tests matter more initially.

3. Master technical flowchart

This is the end-to-end sequence. The next section expands the decision branches for missing information, unsafe suggestions and invalid plans.



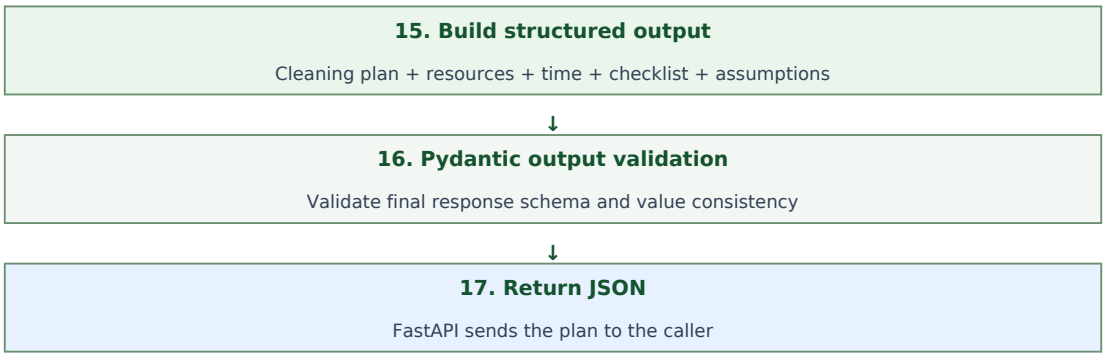


Figure 1. Master sequence. No step should turn unknown information into a claimed fact.

4. Branching flowchart: decision and error handling

The agent must have explicit outcomes for bad requests, missing evidence, unsafe recommendations, unavailable assumptions and invalid plans.

Request structurally valid?

No → return validation error; yes → continue.

↓

Enough information for a useful plan?

No → NEEDS_INFORMATION or NEEDS_REVIEW with missing fields.

↓

Approved template exists?

No → review or controlled fallback; do not invent an authoritative procedure.

↓

Gemini response valid?

No/timeout → controlled failure; retry only where safe; never pretend the call succeeded.

↓

Actions allowed by policy?

No or uncertain → reject proposal or NEEDS_REVIEW.

↓

Resource assumptions available?

No → mark estimate unavailable/preliminary or request data; do not fabricate exact counts.

↓

Duration supported by inputs?

No → label as preliminary/range or request quantity/productivity data.

↓

Final plan complete and consistent?

No → controlled correction or NEEDS_REVIEW; yes → return structured plan.

PLAN_READY	PRELIMINARY_PLAN	NEEDS_REVIEW
Inputs and all required validations support the plan.	Useful plan can be drafted, but estimates rely on stated assumptions or incomplete measurements.	Missing/contradictory evidence, unsafe action or invalid plan blocks safe completion.

5. Step-by-step explanation of the flowchart

Step 1 — Receive the request

The request should include an incident ID, description, category, location, priority and available evidence. The exact schema should be agreed with the caller. Preserve source facts and mark unknowns explicitly.

Step 2 — FastAPI route

The route accepts JSON and calls the service. Keep it thin: it should not contain the cleaning rules, resource arithmetic or prompt construction.

Step 3 — Pydantic input validation

Check field types, allowed category/priority values, numeric ranges and required identifiers. A missing quantity is not zero. Return a clear validation error for malformed requests.

Step 4 — SanitationPlanningService

This is the coordinator. It invokes context preparation, requirement analysis, template selection, AI proposal, resource/time estimators, checklist generation and final validation.

Step 5 — Build planning context

Separate observed information from estimates and unknowns. Record constraints such as blocked access. Do not turn a text inference into a confirmed measurement.

Step 6 — Sufficiency check

Define the minimum facts required for each job type. If a critical safety fact is missing, ask for more information or produce a clearly limited preliminary plan according to policy.

Step 7 — Requirement analyzer

Map the incident to a sanitation job type and required work items. Use deterministic category mappings where possible and AI assistance for ambiguous descriptions. Preserve uncertainty.

Step 8 — Approved template

Load a vetted task template for the job type. Templates reduce inconsistent plans and provide the baseline sequence and permitted actions.

Step 9 — Gemini proposal

Send only relevant incident data and the permitted template/action list. Ask Gemini to adapt steps, explain why and list unknowns. Do not ask it to invent measured quantity, worker availability or authoritative policy.

Step 10 — Action/safety validation

Compare proposed actions against the allowed action list and safety constraints. Reject disallowed actions and escalate ambiguous or high-risk situations.

Step 11 — ResourceEstimator

Calculate required resources from task types, quantity bands, configured productivity assumptions and minimum role requirements. Distinguish resource demand from real-time availability.

Step 12 — TimeEstimator

Estimate each task duration from an explicit rule or historical average. Respect dependencies and parallel tasks. Report assumptions and uncertainty; do not imply precision unsupported by inputs.

Step 13 — ChecklistBuilder

Create stable task IDs, sequence, prerequisites, resource needs and observable completion criteria. A checklist item is a planned step, not proof of completion.

Step 14 — PlanValidator

Check every task has required fields, dependencies are valid, resources cover the planned work, duration is non-negative and all actions are permitted. Check that required tasks are not missing.

Step 15 — Build structured output

Assemble the cleaning objective and steps, resource estimates, duration estimate, checklist, assumptions, warnings and plan status.

Step 16 — Output validation

Pydantic validates the response schema. Additional Python checks enforce semantic consistency—for example, an unknown quantity cannot be represented as a measured value.

Step 17 — Return JSON

FastAPI returns the plan to the caller. The agent has created a plan; it has not executed the job or verified the real-world result.

6. Example request contract

This JSON is illustrative. Final field names and enums should be agreed before the team starts implementing modules.

```
{
  "incident_id": "INC-104",
  "category": "WASTE_OVERFLOW",
  "description": "Overflowing bin and waste partly blocking the path",
  "location": {
    "site_id": "SITE-01",
    "zone": "MAIN_GATE"
  },
  "priority": "MEDIUM",
  "requirements_evidence": {
    "waste_type": "MIXED_MUNICIPAL_WASTE",
    "estimated_quantity_liters": null,
    "path_obstructed": true
  }
}
```

Example Pydantic models

```
from pydantic import BaseModel, Field
from typing import Literal

class LocationInfo(BaseModel):
    site_id: str
    zone: str | None = None

class RequirementsEvidence(BaseModel):
    waste_type: str | None = None
    estimated_quantity_liters: float | None = Field(default=None, ge=0)
    path_obstructed: bool | None = None

class SanitationRequest(BaseModel):
    incident_id: str
    category: str
    description: str
    location: LocationInfo
    priority: Literal["LOW", "MEDIUM", "HIGH"]
    requirements_evidence: RequirementsEvidence
```

The schema deliberately allows missing measurements. The planner must then label the plan or estimate appropriately rather than treating missing data as zero.

7. Requirement analysis and template selection

A first version can map known categories to vetted templates. Gemini can help interpret unusual wording, but the resulting job type and actions must be checked.

```
TEMPLATES = {
    "WASTE_OVERFLOW": [
        "PREPARE_AREA",
        "COLLECT_OVERFLOW",
        "CLEAN_SURROUNDING_AREA",
        "CHECK_ACCESS"
    ],
    "DIRTY_PUBLIC_TOILET": [
        "ASSESS_AREA",
        "CLEAN_APPROVED_SURFACES",
        "REPLENISH_SUPPLIES",
        "CHECK_USABILITY"
    ]
}
```

These are demonstration templates, not approved operational procedures. Real templates should be reviewed by people responsible for sanitation operations and safety.

8. Resource estimation: how the numbers are produced

Resource estimation should be explainable. One simple method is to derive resource requirements from task templates and quantity bands. For example, a task type may specify a minimum crew size and a list of required tools; another rule may increase equipment needs when estimated volume exceeds a configured band.

Illustrative rule data

```
RESOURCE_RULES = {
    "COLLECT_OVERFLOW": {
        "minimum_workers": 2,
        "tools": ["WASTE_COLLECTION_TOOLS"],
        "ppe": ["APPROVED_PROTECTIVE_EQUIPMENT"]
    },
    "CLEAN_SURROUNDING_AREA": {
        "minimum_workers": 1,
        "tools": ["APPROVED_CLEANING_TOOLS"],
        "ppe": ["APPROVED_PROTECTIVE_EQUIPMENT"]
    }
}
```

These counts are placeholders for explaining the software. Do not use them as real-world staffing guidance without operational validation. When tasks run sequentially, the required crew may be the maximum crew needed for a task rather than the sum across every task. When tasks overlap, concurrency and total demand need explicit modelling.

Also keep two questions separate: “What resources does this job require?” and “Which resources are currently available?” The first is planning; the second requires current resource/availability data and is not proven by an estimate.

9. Time estimation: explainable calculations

For a basic model, estimate each task duration using a configured rule, measured quantity and productivity assumption. Example: if a task involves 120 litres and a validated planning assumption is 60 litres/hour, collection time is 120 ÷ 60 = 2 hours. That number is only an example; it is not a universal cleaning rate.

```
collection_hours = quantity_liters / assumed_liters_per_hour
total_duration = (
    setup_minutes
    + collection_hours * 60
    + cleaning_minutes
    + disposal_minutes
)
```

For more complex plans, represent tasks as a dependency graph. If two independent tasks can run at the same time, total elapsed time may be less than the sum of their durations. If a task depends on another, it cannot begin until its prerequisite finishes.

When quantity or productivity data is missing, return a preliminary estimate, a range or NEEDS_INFORMATION according to policy. Do not output a precise number without recording its basis.

10. Checklist data model

```
{
  "task_id": "TASK-02",
  "sequence": 2,
  "description": "Collect overflowing waste",
  "depends_on": ["TASK-01"],
  "required_resources": [
    "WASTE_COLLECTION_TOOLS",
    "APPROVED_PROTECTIVE_EQUIPMENT"
  ],
  "completion_criteria": [
    "Overflowing waste has been collected"
  ],
  "status": "NOT_STARTED"
}
```

Use stable task IDs and validate dependency references. A checklist item should state an observable completion condition. Creating a task does not mean it has been completed.

11. Example output contract

```
{
  "incident_id": "INC-104",
  "planning_status": "PRELIMINARY_PLAN",
  "job_type": "WASTE_OVERFLOW",
  "cleaning_plan": {
    "objective": "Remove overflow and restore access",
    "steps": [
      "Prepare the affected area",
      "Collect the overflowing waste",
      "Clean the surrounding area",
      "Check path access"
    ]
  },
  "resources_required": {
    "workers": {
      "count": null,
      "basis": "QUANTITY_NOT_MEASURED"
    },
    "vehicle_required": null,
    "tools": ["WASTE_COLLECTION_TOOLS"]
  },
  "estimated_time": {
    "minutes": null,
    "estimate_type": "NEEDS_MORE_DATA"
  },
  "task_checklist": [],
  "assumptions": ["Waste quantity has not been measured"]
}
```

This example intentionally uses null for unknown values. It demonstrates that the agent can prepare a partial plan without inventing a worker count or duration.

12. Suggested code layout

```
app/
  main.py
  api/sanitation_routes.py
  schemas/sanitation_request.py
  schemas/sanitation_response.py
  schemas/cleaning_plan.py
  schemas/task_item.py
  services/sanitation_service.py
  services/requirement_analyzer.py
  services/cleaning_plan_builder.py
  services/resource_estimator.py
  services/time_estimator.py
  services/checklist_builder.py
  ai/gemini_client.py
  ai/sanitation_prompt.py
  policies/sanitation_rules.py
  policies/resource_rules.py
  policies/time_estimation_rules.py
  policies/safety_constraints.py
  repositories/in_memory_plan_templates.py
  validators/plan_validator.py

tests/
  test_requirement_analyzer.py
  test_cleaning_plan_builder.py
  test_resource_estimator.py
  test_time_estimator.py
  test_checklist_builder.py
  test_plan_validator.py
  test_sanitation_endpoint.py
```

13. Failure handling and safety

Situation	Expected behavior
Request is malformed	Return validation error; do not call Gemini.
Critical sanitation facts missing	Ask for information or return a limited preliminary plan, according to policy.
No approved template	Use an explicitly approved fallback or request review; do not invent an authoritative procedure.
Gemini timeout/rate limit	Controlled error; no fabricated plan. Retry only when appropriate.
AI proposes a disallowed step	Reject it or require review.
No quantity/productivity data	Mark resource/time estimates unknown or preliminary; state assumptions.
Inconsistent task dependencies	Reject or correct via deterministic rules and revalidate.
Output has missing fields	Fail output validation; do not present it as a valid plan.

14. Testing and acceptance criteria

- A FastAPI endpoint accepts a valid sanitation request and returns structured JSON.
- Pydantic rejects malformed requests and invalid numeric values.
- Requirement analysis distinguishes facts, estimates and unknowns.
- Template selection is deterministic for known categories.
- Gemini output is validated against the allowed actions and schema.
- Resource estimates expose the rules or assumptions that produced them.
- Time estimates do not silently turn unknown quantities into exact durations.
- Checklist items have stable IDs, valid dependencies and completion criteria.
- Plan validation catches missing fields, disallowed actions and inconsistent values.
- Tests cover normal cases, missing data, invalid model responses and API failures.
- The prototype runs without a frontend or persistent database.

Use a mock Gemini client for most automated tests. Run a small number of separate integration tests against the real API. Keep API keys in environment variables and exclude .env from Git.

15. Week 1 build sequence

First: define contracts

Agree on request/response Pydantic schemas, job-type IDs, plan statuses and task structure.

Second: implement deterministic core

Create templates, requirement analysis, resource rules, duration rules, checklist generation and plan validation using synthetic data.

Third: add Gemini

Let Gemini interpret ambiguous descriptions or adapt allowed steps, while validating every proposal.

Fourth: add the FastAPI endpoint

Wire the modules together through the service layer and return JSON.

Fifth: test edge cases

Test missing quantities, unsupported categories, malformed AI output, unsafe suggestions and invalid dependencies.

Final summary

FastAPI receives the request. Pydantic validates it. Python prepares the context. The requirement analyzer identifies the work. An approved template provides the baseline. Gemini can suggest incident-specific adaptations. Python validates actions, estimates resources and duration from explicit rules, builds the checklist and validates the complete plan. The result is returned as structured JSON with unknowns and assumptions clearly marked.